

</ Data Mining: Practical Python for Beginners

Instructor:
Fateme Khodabande

/>

} /> [

fatemever2001@gmail.com

05

Flow of Control & Loops

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

Flow of Control Using if Statements

➡ Comparison Operators

Operator	Description	Example
==	Tests if two values are equal	3 == 3
!=	Tests that two values are <i>not</i> equal to each other	2 != 3
<	Tests to see if the left-hand value is less than the right-hand value	2 < 3
>	Tests if the left-hand value is greater than the right-hand value	3 > 2
<=	Tests if the left-hand value is less than <i>or</i> equal to the right-hand value	3 <= 4
>=	Tests if the left-hand value is greater than or equal to the right-hand value	5 >= 4

Flow of Control Using if Statements

➡ Logical Operators

Operator	Description	Example
and	Returns True if both left and right are true	(3 < 4) and (5 > 4)
or	Returns true if either the left or the right is true	(3 < 4) or (3 > 5)
not	Returns true if the value being tested is False	not 3 < 2

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

Using if Statement

1) Most Basic Form

```
if <condition-evaluating-to-boolean>:  
    statement
```

2) Else in an If Statement

```
if <condition-evaluating-to-boolean>:  
    statement  
else:  
    statement
```

3) The Use of elif

```
if <condition-evaluating-to-boolean>:  
    statement  
elif <condition-evaluating-to-boolean>:  
    statement  
else:  
    statement
```

4) Nesting if Statements

```
if <condition-evaluating-to-boolean>:  
    if <condition-evaluating-to-boolean>:  
        statement
```

5) If Expressions

```
<result1> if <condition-is-met> else <result2>
```

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

Match-Case Statement

Also known as Structural Pattern Matching

```
match <variable>:
  case <value_1> :
    do_something_1()
  case <value_2> :
    do_something_2(second)

    .....
    .....

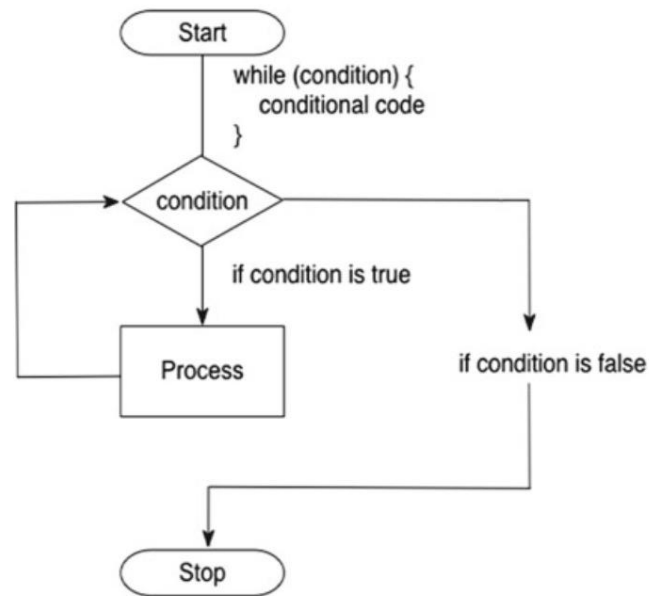
  case _ :
    nothing_matched_function()
```

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

Iteration/Looping [While]

```
while <boolean>:  
    do_something()
```

```
flag = True  
while(flag):  
    if <bool>:  
        flag = False
```



Iteration/Looping (for)

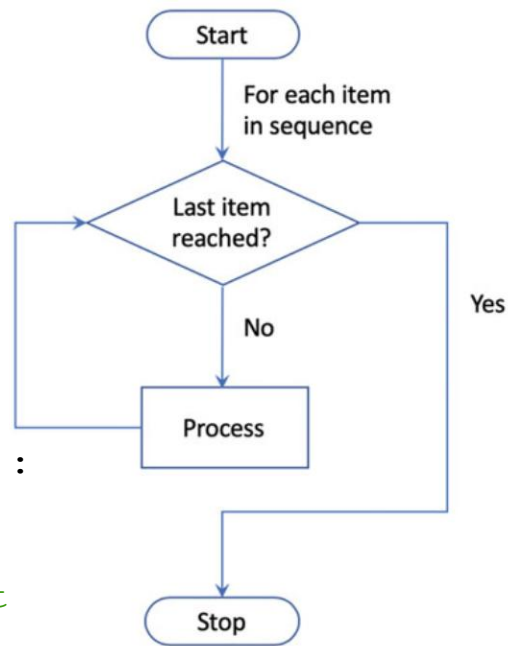
```
for <variable_name> in <iterable>:  
    do_something()
```

<iterable> examples: Tuples, Lists, Sets, Range

```
for i, <variable_name> in enumerate(<iterable>):  
    do_something()
```

```
for <var_1>, <var_2> in zip(<iterable_1>, <iterable_2>):  
    do_something()
```

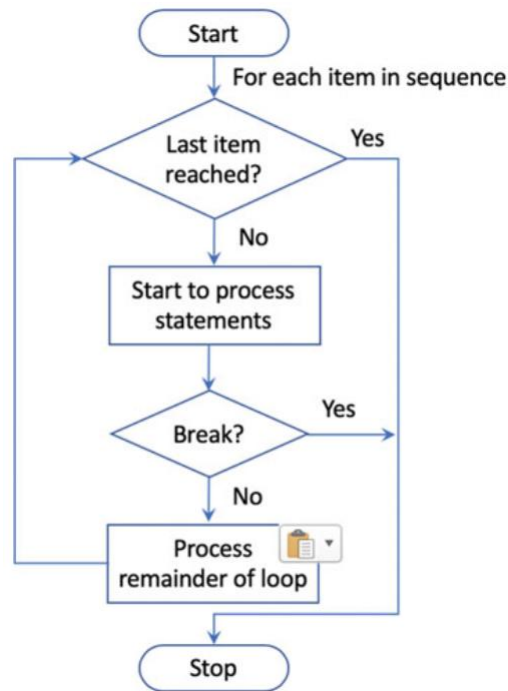
The `zip()` function will only iterate over the smallest list passed



Break Loop Statement

```
print('Only print code if all iterations completed')  
num = int(input('Enter a number to check for: '))
```

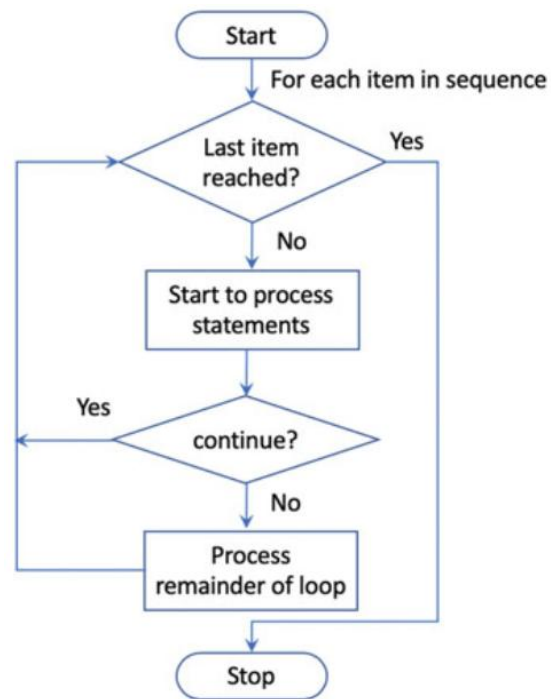
```
for i in range(0, 6):  
    if i == num:  
        break  
    print(i, ' ', end="")  
print('Done')
```



1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

Continue Loop Statement

```
for i in range(0, 10):  
    print(i, ' ', end="")  
    if i % 2 == 1:  
        continue  
    print('hey it is an even number')  
    print('we love even numbers')  
print('Done')
```



1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

For Loop with Else

- A `for` loop can have an *optional* `else` block at the end of the loop.
- The `else` part is executed if and only if all items in the sequence are processed.
- The `for` loop may fail to process all elements in the loop if for some reason an error occurs in your program (e.g., if you have a syntax error) or if you `break` the loop.

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

Collection Related Modules

- A Python list comprehension consists of brackets containing the expression, which is executed for each element along with the for loop to iterate over each element in the Python list.

[<expression> for item in iterable <if optional_condition>]

- The Counter Container
- The Namedtuple Container

06

Functions

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

Functions in Python

- In Python functions are groups of related statements that can be called together, that typically perform a specific task and which may or may not take a set of parameters or return a value.
- Technically speaking there are two types of functions in Python:

built-in functions and user-defined functions.

Defining Functions

- The basic syntax of a function is illustrated below:

```
def function_name([parameter_list]):  
    """docstring"""  
    statement
```

- Arbitrary Arguments



For list

```
def function_name(*args):  
    """docstring"""  
    statement
```



For dictionary

```
def function_name(**kwargs):  
    """docstring"""  
    statement
```

parameter_list and
docstring are optional.

Anonymous Functions

- All the functions we have defined in this chapter have had a name that they can be referenced by.
- This means that we can reference and reuse these functions as many times as we like.
- However, in some cases we want to create a function and use it only once; giving it a name for this one time can pollute the namespace of the program (i.e., there are lots of names around) and also means that someone might call it when we don't expect them to.
- Python therefore has another option when defining a function; it is possible to define an anonymous function. In Python an anonymous function is one that does not have a name and can only be used at the point that it is defined.
- Anonymous functions are defined using the keyword *lambda*, and for this reason they are also known as *lambda functions*.
- `lambda arguments: expression`

Example

```
func1 = lambda x: x * x  
print(func1(4)) -> 16
```


Generators

- In many cases it is not appropriate (or possible) to obtain all the data to be processed up front (for performance reasons, for memory reasons, etc.). Instead of lazily creating the data to be iterated over based on some underlying dataset may be more appropriate.
- Generators are a special function that can be used to generate a sequence of values to be iterated over on demand (that is when the values are needed) rather than produced up front.
- The only thing that makes a generator a generator function is the use of the yield keyword.
- The yield keyword can only be used inside a function or a method. Upon its execution the function is suspended, and the value of the yield statement is returned as the current cycle value. If this is used with a for loop, then the loop runs once for this value. Execution of the generator function is then resumed after the loop has cycled once, and the next cycle value is obtained.
- The generator function will keep supplying values until it returns (which means that an infinite sequence of values can be generated)

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

Recursion

- A recursive solution in a programming language such as Python is one in which a function calls itself one or more times in order to solve a particular problem. In many cases the result of calling itself is combined with the functions current state to return a result
- For example: Computing $n!$ with recursion

```
def factoriel(n):  
    if n == 1:  
        return 1  
    else:  
        return n * factoriel(n-1)  
  
print(factoriel(5))
```

Scope and Lifetime of Variables

- The Global Keyword

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

07

Filter, Map & Sorting

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

Filter

- The `filter()` function is a higher-order function that takes a function to be used to filter out elements from a collection. The result of the `filter()` function is a new *iterable* containing only those elements selected by the test function.
- `filter(function, iterable)`

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

Map

- Map is another higher-order function available in Python. Map applies the supplied function to all items in the *iterable(s)* passed to it. It returns a new *iterable* of the results generated by the applied function.
- It is the functional equivalent of a for loop applied to an *iterable* where the results of each iteration round the for loop are gathered up.
- `map(function, iterable, ...)`

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

Sorting

- There are two built-in ways of sorting within Python: using the `sorted()` function and the `sort()` method.

📌 `sorted_list = sorted(iterable)`

📌 `data_list.sort()`

- Reversing the Order
- Changing the Sort Key
- Sorting Containers of Containers

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

08

Python Modules

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

Python Modules and Packages

- A module allows you to group together related functions, classes and code in general. You can think of a module as being like a library of code (although in fact many libraries are themselves composed of several modules as for example a library may have optional or extensions to the core functionality).

- Importing from a module

```
from <module name> import *
```

- Importing Aliases

```
import <module_name> as <alternative_module_name>  
from <module_name> import <element> as <alias>
```

- Standard Modules
- Python allows developers to organize modules together into packages, in a hierarchical structure based on directories

Exercise 3

You are given a simple list of daily sales values that contain missing (None) or zero entries.

```
[250, None, 0, 430, 120, 510, None, 300, 0, 400]
```

Your task is to:

- 1) Filter out invalid or missing values using `filter()` and a lambda function.
- 2) Transform the cleaned data by applying a 90% discount using `map()`.
- 3) Sort the resulting list in descending order of sales.
- 4) Define a function that returns a summary dictionary with count, minimum, maximum, and average sales.
- 5) Print all results step by step.

```
1 0 1 1    0 1 1    0 1    1 0 1 1 0 0 1    1 0    1 1 0 1 1    0 1 1    0 1    1 1 0 1 1 0    1 1 0 1 1 1    1 1 0 1
```

</ Reference: A Beginners Guide to Python 3 Programming />

<https://doi.org/10.1007/978-3-031-35122-8>

